Architecture Specification

Agentic Access to ATLAS DAQ/OKS Historical Configuration

Natural Language to OksQuery

Architecture Type:	Agentic configuration-query system
Primary Interface:	MCP Server
Implementation:	Python
Configuration Backend:	TDAQ `config` / `oksconfig`
Configuration Engine:	OKS
Query Language:	OksQuery
Access Mode:	Read-only
Primary Historical Store:	ATLAS OKS Git repository

Architecture Design Document

Version 1.0

Contents

1 Document Purpose

This document specifies the proposed prototype architecture for **Agentic Access to ATLAS DAQ/OKS Historical Configuration**.

The system provides a natural-language interface through which a user can ask questions about the configuration of the ATLAS DAQ system at a particular historical run.

The central architectural objective is:

> **Natural Language → Intent → Historical Run Resolution → Validated Revision → Existing TDAQ Configuration Stack → Existing OKS Kernel → Schema Context → OksQuery Generation → Native OksQuery Validation → Existing OKS Execution → Result Serialization → Natural Language Answer**

The architecture deliberately treats the existing TDAQ and OKS software as the configuration and query engine. The new system does not attempt to replace OKS, implement another configuration engine, parse the OKS XML files independently, or implement a second OksQuery parser. Instead, the new application provides the semantic and orchestration layer around the existing infrastructure.

2 Architectural Objectives

The prototype has the following objectives.

2.1 Natural-Language Access

The system shall allow a user to express a configuration question using natural language. For example, a user may ask a question conceptually equivalent to:

> "Which object of class X was configured with property Y in run R?"

The system converts the question into a structured intent and then into an OksQuery that can be evaluated against the historical configuration.

2.2 Historical Configuration Access

The system shall resolve a run number to the corresponding historical configuration revision. The preferred mechanism is the existing run-tag convention:

```
r<run>@<partition>
```

For example:

```
r123456@ATLAS
```

The tag identifies the configuration revision associated with the run and partition. If the run tag cannot be resolved, the prototype may use the Run Number database as a fallback.

2.3 Reuse of Existing TDAQ Infrastructure

The prototype shall reuse existing components wherever possible. Important existing components include:

- `libpyconfig`
- `config::Configuration`

- `oksconfig`
- `OksConfiguration`
- `OksKernel`
- `Configuration::get_class_info()`
- `Configuration::subclasses()`
- `OksQuery`
- `OksQuery::good()`
- `OksClass::execute_query()`
- `conf.get_objs()`
- Existing OKS Git integration

No custom replacement should be created for these capabilities.

2.4 Read-Only Operation

The prototype shall operate as a read-only service. The intended operation is:

$$\text{User} \rightarrow \text{Application} \rightarrow \text{TDAQ/OKS read APIs} \rightarrow \text{Historical Configuration}$$

There shall be no application path that performs:

- configuration mutation,
- `commit()`,
- `abort()`,
- `set_config_version()`,
- Git push,
- configuration creation,
- configuration deletion,
- RDB writer operations.

The presence of write APIs in the Python binding does not change this architectural decision. Read-only behaviour must be enforced by the application design and deployment permissions.

2.5 Revision-Correct Querying

A generated query must be validated against the actual schema loaded for the historical revision being queried. This is important because a class, attribute, relation, or type may change between configuration revisions. Therefore:

$$\boxed{\text{Resolve Revision} \rightarrow \text{Load Revision} \rightarrow \text{Retrieve Schema} \rightarrow \text{Generate Query} \rightarrow \text{Validate Query}}$$

rather than generating a query against a generic or current schema.

3 Architectural Principles

3.1 Principle 1 — Existing TDAQ/OKS Components Remain the Authority

The prototype does not become the source of truth for configuration. The authoritative configuration remains the historical OKS configuration accessed through the existing TDAQ configuration stack. The LLM is therefore not allowed to invent configuration facts. The LLM proposes a query; the existing OKS system determines the actual result.

3.2 Principle 2 — LLM as Semantic Layer, Not Infrastructure Layer

The local LLM performs semantic tasks such as:

- understanding the user's question,
- identifying entities and concepts,
- selecting relevant schema information,
- generating OksQuery syntax,
- repairing invalid generated queries,
- producing a natural-language explanation of returned results.

The LLM must not directly perform infrastructure operations. In particular, the LLM shall not directly access:

- Git,
- the filesystem,
- the shell,
- OKS,
- the Run Number database,
- TDAQ services.

All infrastructure access is performed by controlled application modules.

3.3 Principle 3 — Native Validation

The system shall not implement a custom OksQuery validator. The generated query is passed to the native:

```
OksQuery(class_name, query_text)
```

and validated through:

```
good()
```

This provides validation against the actual OKS query implementation and the loaded configuration schema.

3.4 Principle 4 — No Independent XML Interpretation

The application shall not independently parse the OKS XML files to reconstruct the configuration. The existing OKS kernel already handles:

- configuration loading,
- include resolution,
- schema loading,
- object loading,
- inheritance,
- Git-backed configuration revisions.

Reimplementing these mechanisms would introduce unnecessary duplication and potentially produce behaviour different from the actual TDAQ system.

3.5 Principle 5 — Historical Revision Before Schema

Schema retrieval occurs only after the historical configuration has been resolved and loaded. The schema is therefore tied to:

$$(\text{revision}, \text{partition}, \text{class})$$

rather than being treated as a global static resource.

3.6 Principle 6 — Controlled Fallbacks

Fallback mechanisms shall not become hidden alternative primary paths. The primary historical-resolution path is:

$$\text{Run Number} + \text{Partition} \rightarrow r\langle run\rangle@\langle partition\rangle \rightarrow \text{Git Revision}$$

The Run Number database is explicitly a fallback. The `roksconfig` component is also optional. Its use depends on confirmation that the historical CORAL/Oracle archive remains available.

4 System Context

4.1 External User

The user is the operator or researcher asking a historical configuration question. The user communicates with the system through the MCP interface. The user does not directly interact with:

- OKS,
- Git,
- the Run Number database,
- TDAQ configuration APIs.

4.2 MCP Server

The MCP server is the application-facing entry point. Its responsibility is to:

- receive a request,
- invoke the appropriate application workflow,
- return the final structured or natural-language response.

The MCP server is not itself the configuration engine. It orchestrates the application modules that perform the actual historical configuration access and query processing.

4.3 ATLAS OKS Git Repository

The Git repository is the primary historical configuration source. It contains versioned OKS configuration data and schema. The application uses the repository in read-only mode. The prototype does not push changes to the repository.

4.4 Run Number Database

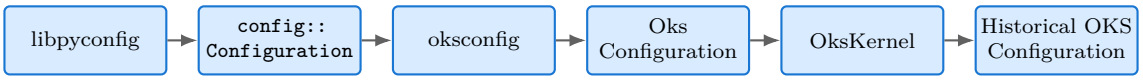
The Run Number database is a fallback source for resolving a run to configuration metadata. The fallback may provide information such as:

- configuration version,
- configuration name,
- partition name.

The Run Number database is deliberately not placed on the primary execution path.

4.5 TDAQ Configuration Stack

The existing TDAQ configuration stack provides the controlled interface between the application and OKS. The relevant logical stack is:



This stack is reused instead of recreated.

5 High-Level Architecture

The system consists of the following logical stages:

1. MCP request handling
2. Intent extraction
3. Historical configuration resolution
4. Revision validation
5. Historical configuration loading
6. Schema retrieval

7. Query-generation context construction
8. OksQuery generation
9. Native query validation
10. Bounded query repair
11. Query execution
12. Result limiting
13. Result formatting
14. Natural-language answer generation
15. MCP response

The primary data flow is illustrated in fig. 1.

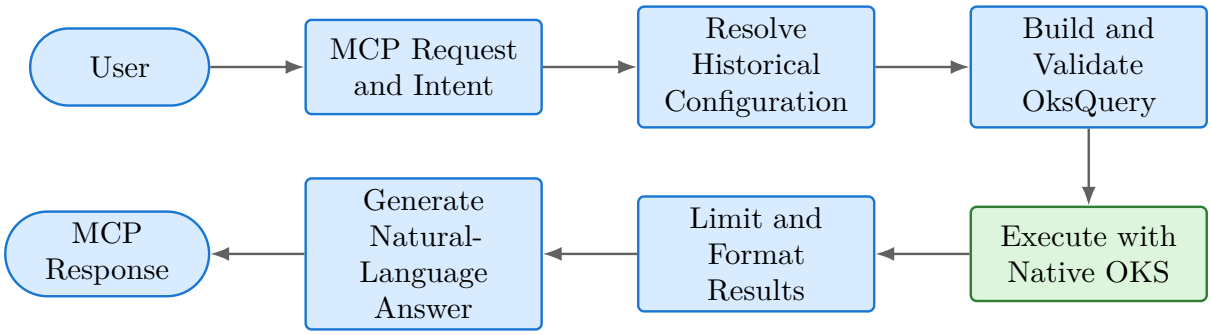


Figure 1: Primary end-to-end architecture flow

An version of the main architecture diagram is available in Whimsical.

6 Module Classification

Each component is classified according to whether it is newly implemented by the prototype or reused from the existing TDAQ/OKS ecosystem.

6.1 New Application Components

The prototype introduces the following logical modules:

- Intent Extractor
- Historical Configuration Resolver
- Run Tag Resolver
- Revision / Version Validator
- Schema Context Builder
- Query Generation Context Builder
- Local LLM integration

- OksQuery Generator
- Query Repair Controller
- Result Limiter
- Result Formatter / Serializer
- Answer Generator
- Error Handler

These components form the new semantic and orchestration layer.

6.2 Existing Components

The following components are reused:

- `libpyconfig`
- `config::Configuration`
- `oksconfig`
- `OksConfiguration`
- `OksKernel`
- OKS Git integration
- `Configuration::get_class_info()`
- `Configuration::subclasses()`
- `OksQuery`
- `OksQuery::good()`
- `OksClass::execute_query()`
- `conf.get_objs()`
- OKS object APIs

The application must use these interfaces rather than replacing them.

6.3 Optional Components

The following components are retained as optional or fallback paths:

- Run Number database
- `roksconfig`
- CORAL/Oracle historical OKS archive
- `rdbconfig`
- CORBA
- RDB server

These are not part of the primary six-week execution path unless required by the deployment environment or confirmed operational requirements.

7 Detailed Module Architecture

The preceding sections define the system goals, boundaries, context, and high-level flow. The remainder of this document develops that architecture module by module, following the same processing order from the user-facing interface to historical configuration access, query execution, and the final response.

The architecture is designed around the principle that the prototype should add semantic intelligence around the existing TDAQ/OKS configuration infrastructure rather than replace any part of that infrastructure.

The overall processing principle is:

> **Natural Language → Intent → Historical Run Resolution → Validated Revision → Existing TDAQ Configuration Stack → Existing OKS Kernel → Schema Context → OksQuery Generation → Native OksQuery Validation → Existing OKS Execution → Result Serialization → Answer Generation → MCP Response**

The application is strictly read-only. No module in the prototype is permitted to modify an OKS configuration, create a commit, push to a Git repository, or modify production configuration state.

7.1 Architectural Layering

The system can be divided into the following logical layers:

1. Client and protocol layer
2. Natural-language understanding layer
3. Historical configuration resolution layer
4. Configuration access layer
5. Schema understanding layer
6. Query generation layer
7. Native query validation layer
8. Query execution layer
9. Result processing layer
10. Natural-language answer layer
11. Error handling layer

The distinction between these layers is important because the LLM should not become responsible for infrastructure operations. The LLM performs semantic interpretation and query generation, while the application and existing TDAQ/OKS libraries perform configuration access, validation, and execution.

8 Module 1 – User / Operator

8.1 Purpose

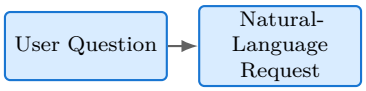
The User / Operator is the external source of a natural-language request. The user asks questions about historical ATLAS DAQ/OKS configuration. Example questions include:

- What configuration was used for run 123 456?
- Which processes were configured in partition P1 during run 123 456?
- What was the configuration of the XYZ component in run 123 456?
- Show the value of attribute A for all objects of class B in run 123 456.

The user does not need to know OksQuery syntax.

8.2 Input

The input is a natural-language question. Conceptually:



8.3 Output

The user-facing system eventually returns a natural-language answer containing structured provenance information. The final response should normally expose:

- interpretation of the question;
- run number;
- partition;
- resolved configuration revision;
- answer or result set;
- generated OksQuery;
- truncation information, if applicable.

8.4 Responsibilities

The User / Operator module has no responsibility for:

- Git operations;
- configuration loading;
- schema parsing;
- OksQuery validation;
- query execution;
- configuration modification.

It is simply the external consumer of the system.

9 Module 2 – MCP Server

9.1 Purpose

The MCP Server is the application-facing protocol boundary of the prototype. It exposes the historical configuration query capability to an MCP-compatible client. The MCP server should be considered the entry and exit point of the application rather than the component responsible for performing the actual TDAQ/OKS operations.

9.2 Input

The MCP server receives a request containing the user's question and, depending on the client protocol, associated request metadata. Conceptually:

MCP request fields:
- natural-language question;
- optional run number and partition;
- request metadata.

9.3 Output

The MCP server returns the final structured response. Conceptually:

MCP response fields:
- interpretation, run number, partition, and revision;
- generated query and structured results;
- natural-language answer and truncation metadata.

9.4 Responsibilities

The MCP server is responsible for:

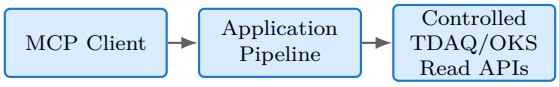
1. receiving requests;
2. invoking the application pipeline;
3. returning structured responses;
4. exposing errors in a controlled format;
5. preventing direct access from the client to internal TDAQ components.

9.5 Security Boundary

The MCP server forms part of the application's trust boundary. The following direct paths must not exist:

The MCP client must not directly access Git, the filesystem, OksKernel, configuration-mutation APIs, or the shell.

Instead:



10 Module 3 – Intent Extractor

10.1 Purpose

The Intent Extractor converts the natural-language question into a structured representation that can be processed deterministically by the rest of the system. This module is one of the new application-level components. Its primary purpose is to separate semantic interpretation from infrastructure resolution.

10.2 Input

The Intent Extractor receives:

- original user question;
- optional request metadata supplied by the MCP client.

Example:

```
"What processes were configured in partition P1 during run 123456?"
```

10.3 Output

The module produces a structured intent. A conceptual representation is:

```
{
    run_number: 123456,
    partition: "P1",
    concept: "processes",
    filters: [...],
    requested_fields: [...],
    operation: "retrieve"
}
```

The exact implementation representation may be JSON, Python objects, dataclasses, or another internal representation.

10.4 Intent Fields

The structured intent should contain, where applicable:

`run_number`	The requested historical run number.
`partition`	The ATLAS/TDAQ partition associated with the request.
`concept`	The conceptual entity the user is asking about.
`filters`	Constraints expressed by the user.
`requested_fields`	Attributes or properties requested by the user.
`operation`	The type of read operation requested.

10.5 Important Boundary

The Intent Extractor should extract the run number. It must **not** decide which Git revision corresponds to the run. For example:

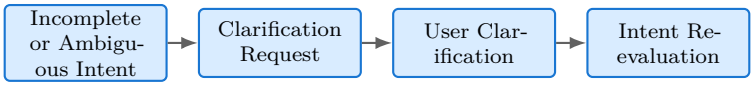
This separation prevents the LLM from inventing or guessing historical configuration revisions.

10.6 Failure Conditions

The Intent Extractor may be unable to produce a complete, reliable intent if:

- the question is ambiguous;
- no meaningful configuration-related intent can be extracted;
- the requested operation is unsupported;
- required information is missing.

In these cases, the system must not guess missing values, select a likely interpretation, or continue with assumed defaults. Instead, the Intent Extractor returns a clarification-required result through the MCP server. The response should briefly state what was understood, identify the ambiguous or missing information, and ask one focused follow-up question.



The user's clarification is combined with the original request and passed through intent extraction again. If the requested operation is outside the prototype's supported scope, the response should explain the limitation and invite the user to reformulate the question. The central Error Handler is used only when clarification cannot resolve the request or when a non-semantic system failure occurs.

11 Module 4 – Historical Configuration Resolver

11.1 Purpose

The Historical Configuration Resolver determines which historical configuration revision corresponds to the requested run. This module is critical because the system is intended to answer questions about historical configurations rather than merely the current configuration.

11.2 Input

The resolver receives:

- run number;
- partition.

For example:

```
run_number = 123456
partition = P1
```

11.3 Primary Resolution Mechanism

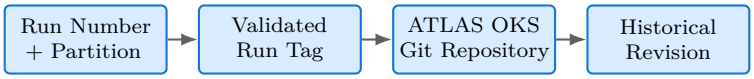
The primary resolution mechanism is the run-specific Git tag:

```
r<run>@<partition>
```

For example:

```
r123456@P1
```

The resolver therefore performs:



11.4 Why Git Tags Are Preferred

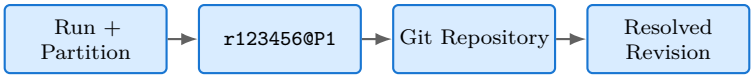
The repository investigation established that run allocation creates run-specific tags following this convention. Therefore, the resolver does not need to infer a revision from the configuration contents. It asks the repository for the revision associated with the known run-specific tag. This makes historical resolution deterministic.

11.5 Git Tag Resolver

The Historical Configuration Resolver delegates the primary lookup to a Git Tag Resolver. The Git Tag Resolver:

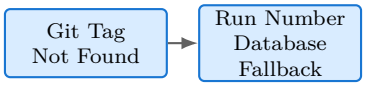
1. receives a run number;
2. receives a partition;
3. constructs the expected tag identifier;
4. validates the identifier;
5. queries the repository;
6. returns the resolved revision.

Conceptually:



11.6 Git Tag Not Found

If the expected tag cannot be found, the system must not simply guess another revision. Instead, it activates the fallback path.



12 Module 5 – Run Number Database Fallback

12.1 Purpose

The Run Number Database provides a fallback mechanism for historical configuration resolution when the expected Git run tag cannot be found. This is explicitly a fallback path and is not part of the primary execution path.

12.2 Input

The fallback receives:

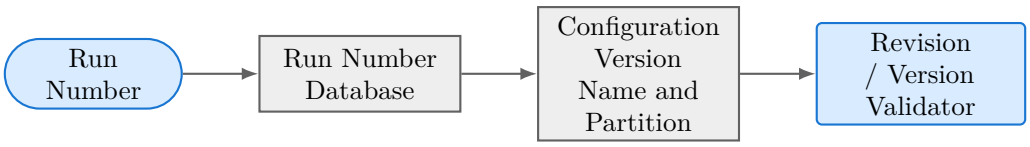
- run number;
- partition where required.

12.3 Retrieved Information

The database may provide:

- configuration version;
- configuration name;
- partition information;
- historical configuration references.

The architecture should represent the fallback as:



12.4 Architectural Rule

The Run Number Database must not be treated as the primary historical configuration store. Its role is to resolve a run when the preferred run-tag mechanism is unavailable.

12.5 Fallback Label

In the architecture diagram this path should be labelled `FALLBACK -- Run Number DB` and represented using a dashed connection.

13 Module 6 – Revision / Version Validator

13.1 Purpose

The Revision / Version Validator is a security-critical boundary between historical resolution and configuration loading. Its purpose is to ensure that a revision identifier is valid before it is supplied to the TDAQ/OKS configuration-loading infrastructure.

13.2 Why Validation Is Required

The repository investigation found that configuration operations ultimately reach shell commands through calls involving `system()`. Consequently, a revision identifier must never be treated as arbitrary untrusted text. The LLM must never be allowed to generate an unrestricted revision string that is subsequently passed to configuration infrastructure.

13.3 Input

The validator receives a resolved version identifier from:

- Git Tag Resolver; or
- Run Number Database fallback.

13.4 Validation

The validator should enforce a strict grammar appropriate for each type of identifier.

For Git hashes, the prototype can use a restricted form such as:

```
^[0-9a-f]{7,40}$
```

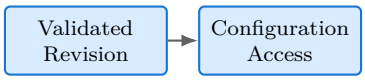
For run tags, the expected structure is:

```
^r[0-9]+@[A-Za-z0-9_-]+$
```

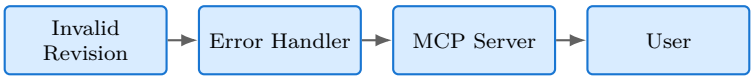
The exact accepted grammar should follow the production conventions confirmed with the TDAQ experts.

13.5 Output

If valid:

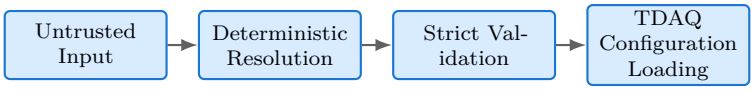


If invalid:



13.6 Security Principle

No generated natural-language content should be passed directly into `TDAQ_DB_VERSION`. The required flow is:



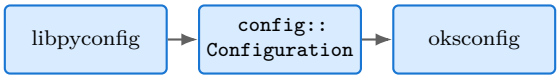
14 Module 7 – TDAQ Configuration Access Layer

14.1 Purpose

The TDAQ Configuration Access Layer is the application's controlled interface to the existing TDAQ configuration subsystem. This layer prevents higher-level application logic from depending on low-level implementation details.

14.2 Primary Technology

The prototype uses:



The application should reuse these existing components rather than implementing an alternative configuration engine.

14.3 Responsibilities

The access layer is responsible for:

- selecting the appropriate configuration backend;
- loading the historical configuration;
- exposing configuration APIs to the application;
- providing access to schema information;
- providing read-only query execution.

It should not implement:

- XML parsing;
- Git checkout logic;
- OksQuery parsing;
- OksQuery validation;
- custom configuration persistence.

15 Module 8 – libpyconfig

15.1 Purpose

`libpyconfig` provides the Python binding through which the Python-based prototype communicates with the TDAQ configuration subsystem. This is the preferred application integration point.

15.2 Flow

The primary path is:

15.3 Why It Is Reused

The repository investigation confirmed that Python bindings already exist. Reimplementing the configuration interface would duplicate existing TDAQ functionality and introduce unnecessary compatibility risks.

15.4 Important Security Consideration

The Python binding exposes both read and write operations. Importing the `libpyconfig` module therefore does not guarantee read-only behaviour. The prototype must enforce read-only operation through:

1. deployment permissions;
2. application-level API discipline;

3. omission of all mutation operations from the prototype;
4. repository permissions that do not allow pushes.

The architecture therefore treats `libpyconfig` as an existing technology component but constrains its use to the read path.

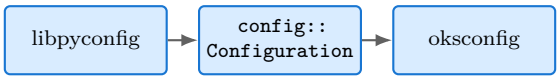
16 Module 9 – config::Configuration

16.1 Purpose

`config::Configuration` is the technology-neutral configuration interface used by the application. It provides the abstraction between the Python application and the specific configuration backend.

16.2 Primary Path

The prototype uses:



The important architectural property is that backend selection happens through the existing configuration plugin mechanism.

16.3 Backend Independence

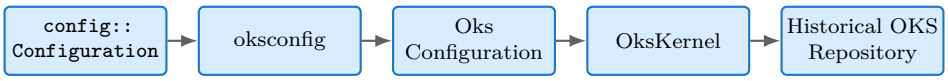
The architecture does not hard-code the entire application around `oksconfig`. The backend can theoretically be changed below `config::Configuration`. This is important because the repository investigation identified multiple existing configuration backends.

17 Module 10 – oksconfig

17.1 Purpose

`oksconfig` is the primary backend selected for the first prototype. It connects the generic configuration interface to the OKS kernel and historical OKS repository.

17.2 Primary Flow



17.3 Responsibilities

The backend is responsible for:

- loading the requested configuration;
- interacting with the OKS kernel;
- resolving repository data;
- providing configuration objects;
- forwarding OksQuery operations.

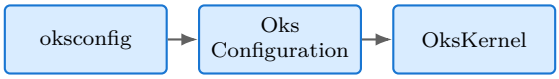
The prototype does not reimplement these functions.

18 Module 11 – OksConfiguration

18.1 Purpose

`OksConfiguration` is the existing OKS configuration implementation used by the `oksconfig` backend. It connects the generic configuration interface to the underlying `OksKernel`.

18.2 Flow



The application should interact with this functionality through the existing configuration API rather than creating an alternative OKS configuration implementation.

19 Module 12 – OksKernel

19.1 Purpose

`OksKernel` is the existing OKS engine responsible for loading the configuration into memory and maintaining the in-memory classes and objects. It is one of the most important existing components in the architecture.

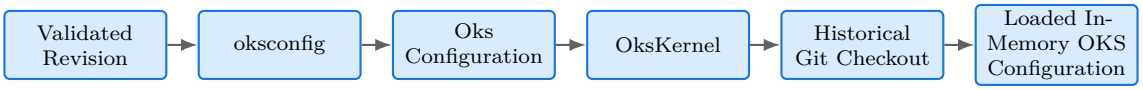
19.2 Responsibilities

The kernel handles functionality such as:

- loading OKS configuration;
- repository interaction;
- Git-based revision handling;
- include resolution;
- class loading;
- object loading;
- maintaining the in-memory configuration model.

19.3 Historical Configuration

The prototype does not directly parse historical XML files. Instead:



This is an important architectural principle. The prototype delegates historical repository handling to the existing OKS/TDAQ implementation.

20 Module 13 – ATLAS OKS Git Repository

20.1 Purpose

The ATLAS OKS Git Repository is the external source of versioned historical configuration. It stores the versioned configuration data used by the OKS system.

20.2 Access

The repository is accessed through the existing TDAQ/OKS Git integration. The application should not implement its own Git checkout mechanism. The desired architecture is:

The actual repository endpoint used by the deployment must be confirmed with the ATLAS/TDAQ experts.

20.3 Read-Only Requirement

The prototype only requires repository read access. There must be no application path such as:

> The application must not commit to Git, push repository changes, or modify configuration state.

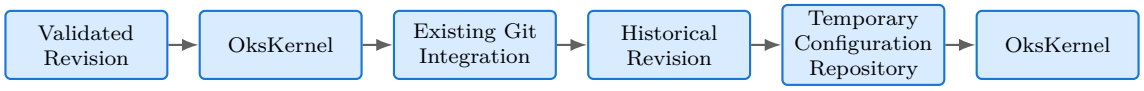
The prototype should operate using an identity that cannot modify the production repository.

21 Module 14 – Historical Git Checkout

21.1 Purpose

The Historical Git Checkout is performed by the existing TDAQ/OKS infrastructure. The application requests a specific historical revision, and the existing OKS infrastructure obtains the corresponding configuration.

21.2 Flow



The application must not introduce a custom Git implementation.

21.3 Temporary Repository

The existing OKS mechanism may create a temporary user repository for the requested revision. The prototype must ensure that temporary resources are eventually released. This is an infrastructure concern rather than a custom caching system.

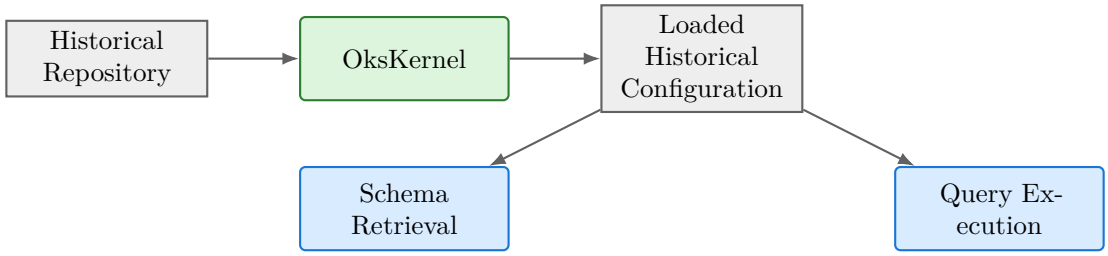
22 Module 15 – Loaded In-Memory OKS Configuration

22.1 Purpose

Once the historical configuration has been loaded, the OksKernel contains the classes and objects corresponding to that historical revision. This becomes the source for both:

1. schema retrieval;
2. query execution.

22.2 Flow



The schema and data are therefore tied to the same historical revision. This is important for correctness because the query generator must not generate a query against a schema from a different revision.

23 Module 16 – Schema Retriever

23.1 Purpose

The Schema Retriever obtains the schema information required to understand the historical configuration. The schema must be retrieved from the same loaded historical configuration that will eventually execute the generated query.

23.2 Existing APIs

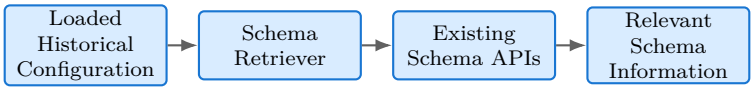
The prototype reuses existing configuration APIs including:

- `Configuration::get_class_info()`;
- `Configuration::subclasses()`.

Additional existing class information APIs can expose:

- classes;
- attributes;
- attribute types;
- relations;
- relation targets;
- inheritance information.

23.3 Flow



23.4 Why Schema Retrieval Is Required

The LLM cannot reliably generate an OksQuery merely from the natural language question. It must know the actual schema of the historical configuration. For example, if the user asks:

```
"Find all processes with state X"
```

the generator needs to know:

- which class represents a process;
- which attribute represents state;
- the type of the state attribute;
- the exact attribute identifier;
- relevant inheritance relationships.

24 Module 17 – Schema Context Builder

24.1 Purpose

The Schema Context Builder converts retrieved schema information into a compact context suitable for query generation. The complete OKS database should not automatically be sent to the LLM.

24.2 Input

The module receives:

- structured intent;
- available schema information.

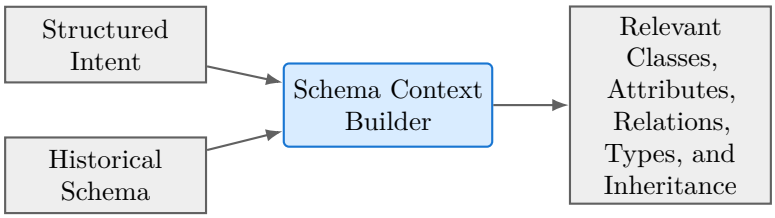
24.3 Output

It produces:

```
Relevant Schema Context
```

The context should contain only schema information relevant to the current question.

24.4 Conceptual Flow



This reduces unnecessary LLM context and reduces the probability of selecting unrelated classes.

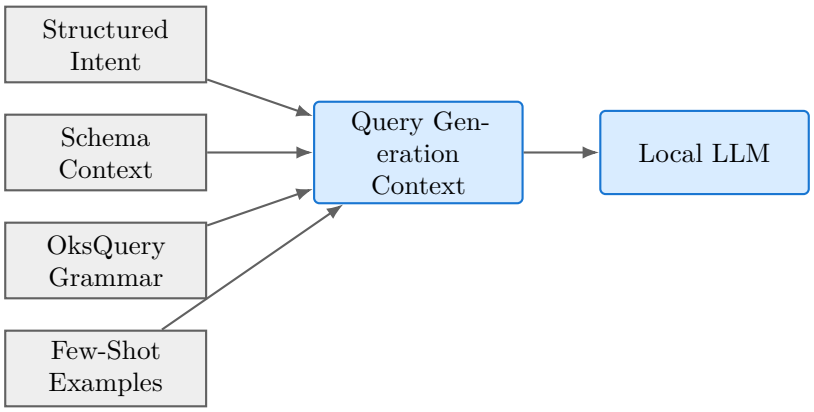
25 Module 18 – Query Generation Context

25.1 Purpose

The Query Generation Context is the complete controlled context supplied to the local LLM for OksQuery generation. It combines four major inputs:

1. structured intent;
2. relevant schema context;
3. OksQuery grammar;
4. few-shot examples.

25.2 Architecture



The context builder is responsible for ensuring that the model receives the information necessary to construct a valid query without receiving unnecessary infrastructure details.

26 Module 19 – LLM

26.1 Purpose

The LLM provides semantic reasoning for:

- interpreting user intent;
- mapping natural-language concepts to schema entities;
- generating OksQuery expressions;
- generating natural-language answers from structured results.

The LLM is an application component, not an OKS replacement.

26.2 Strict Access Boundary

The LLM must not directly access:

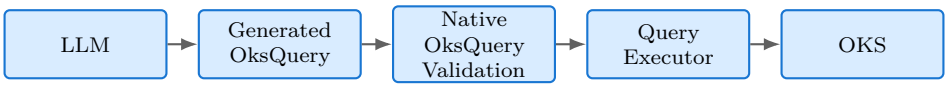
- Git;
- filesystem;

- shell;
- OksKernel;
- Run Number Database;
- TDAQ infrastructure.

The application remains the controlled intermediary.

26.3 Required Flow

The LLM must follow:



There must not be:

> The LLM must not directly access the shell, Git, the filesystem, or OKS.

27 Module 20 – OksQuery Generator

27.1 Purpose

The OksQuery Generator converts the structured query-generation context into an OksQuery request. This is one of the primary new components of the prototype.

27.2 Input

The generator receives:

- structured intent;
- relevant schema context;
- OksQuery grammar;
- few-shot examples.

27.3 Output

The generator should produce:

```
(class_name, oksquery_text)
```

For example:

```
class_name:
SomeClass

query:
("attribute" "value" =)
```

The exact query must be generated according to the actual schema and OksQuery grammar.

27.4 Important Architectural Rule

The generator does not execute the query. It only proposes a query. The proposed query must pass through the existing native OksQuery validation mechanism before execution.

28 Module 21 – Native OksQuery

28.1 Purpose

The generated query is passed to the existing native OksQuery implementation. The prototype must not implement another query engine.

28.2 Flow

The native implementation becomes the authoritative mechanism for determining whether the generated query is valid.

29 Module 22 – OksQuery.good() Validator

29.1 Purpose

`OksQuery.good()` is reused as the query validation mechanism. This is a major architectural decision. The prototype should not create a custom OksQuery parser or validator.

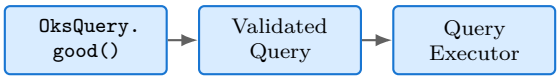
29.2 Validation

The existing validation mechanism checks the generated expression against the actual OKS schema and query rules. The validation therefore occurs against the same historical configuration that will execute the query.

29.3 Decision

The architecture contains two branches.

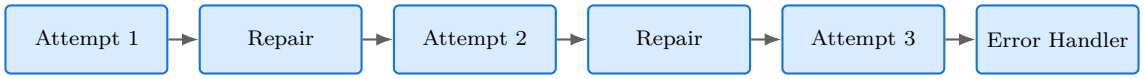
29.3.1 Valid Query



29.3.2 Invalid Query

29.4 Bounded Repair

The repair mechanism should be bounded to approximately two or three attempts. The purpose of bounded repair is to correct deterministic query errors without allowing an uncontrolled model loop. Conceptually:

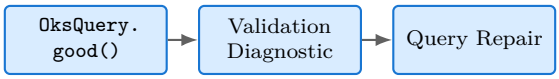


30 Module 23 – Validation Diagnostic

30.1 Purpose

The Validation Diagnostic contains information explaining why the generated query failed validation. The diagnostic is provided to the bounded repair process.

30.2 Flow



The diagnostic should not be treated as a substitute for the native validator. The native validator remains authoritative.

31 Module 24 – Bounded Query Repair

31.1 Purpose

The Bounded Query Repair module attempts to correct an invalid generated query. It receives:

- original structured intent;
- relevant schema context;
- generated query;
- validation diagnostic.

It then requests a corrected query from the LLM.

31.2 Important Constraint

The repair loop must remain bounded. A suitable prototype limit is:

$$N_{\max} = 3$$

where $N_{\max}$ represents the maximum number of validation attempts.

31.3 Failure

If the query remains invalid after the retry limit:

32 Module 25 – Query Executor

32.1 Purpose

The Query Executor executes a validated OksQuery against the loaded historical configuration. It reuses the existing OKS execution mechanism.

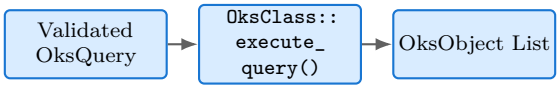
32.2 Existing Execution API

The architecture uses the existing mechanisms represented by:

```
OksClass::execute_query()
```

The executor receives only queries that have passed native validation. It returns the matching OKS objects to the result-processing pipeline.

32.3 Flow



33 Module 26 – Result Limiter

33.1 Purpose

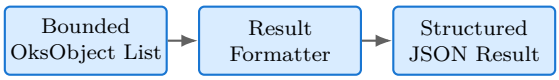
The Result Limiter applies deterministic bounds before results are serialized or sent to the LLM. It records whether the returned result set was truncated and, where available, the total number of matches.

34 Module 27 – Result Formatter / Serializer

34.1 Purpose

The Result Formatter converts native OKS objects into a stable, JSON-compatible representation containing requested attributes, identifiers, class information, and historical provenance.

34.2 Flow



35 Module 28 – Answer Generator

35.1 Purpose

The Answer Generator uses the structured result and provenance metadata to produce the final natural-language response. It must not add configuration facts that are absent from the executed query results.

36 Module 29 – Central Error Handler

36.1 Purpose

The Central Error Handler converts failures from every stage into controlled, structured errors. It distinguishes invalid input, unresolved revisions, configuration-loading failures, invalid queries, retry exhaustion, and execution failures.

37 Module 30 – MCP Response Builder

37.1 Purpose

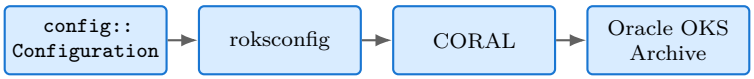
The MCP Response Builder assembles the interpretation, run and partition, resolved revision, generated query, bounded results, truncation metadata, answer, and any structured error information for return to the client.

38 Module 31 – roksconfig Backend

38.1 Purpose

The existing `roksconfig` backend provides read-only access to a historical CORAL/Oracle archive. It is retained only as an optional path whose operational availability must be confirmed.

38.2 Architecture



38.3 Why Optional

The archive's operational status must be confirmed before it becomes part of the prototype. The architecture labels this path `OPTIONAL -- only if the historical archive is confirmed.`

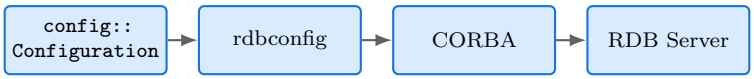
39 Module 32 – rdbconfig Backend

39.1 Purpose

`rdbconfig` is an existing TDAQ backend based on CORBA and a remote RDB server. It is deliberately excluded from the primary prototype architecture.

39.2 Existing Architecture

Conceptually:



39.3 Why It Is Not Used

The repository investigation found that changing the configuration version through the RDB path can affect shared server state. This creates an undesirable architecture for a read-only query service. Therefore:

> `rdbconfig` is explicitly excluded from the primary historical query path.

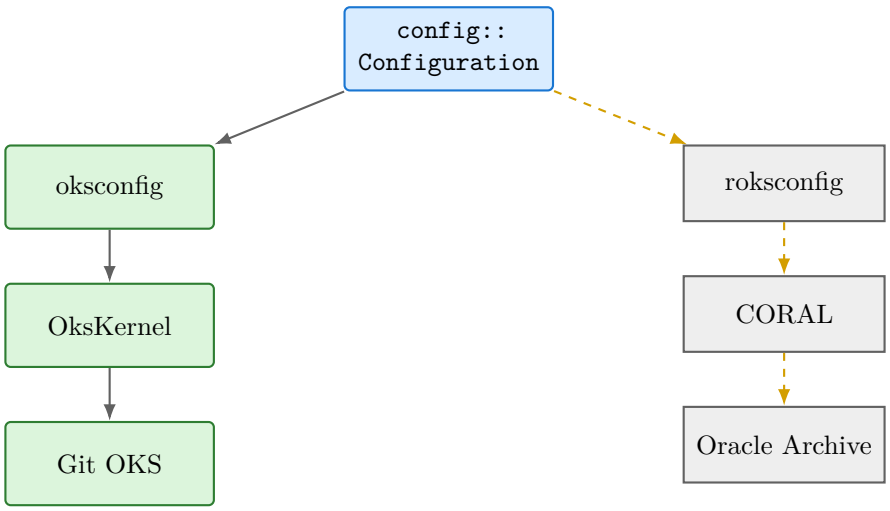
It may be shown in the architecture as an existing but unused backend.

40 Module 33 – Configuration Access Backend Selection

40.1 Purpose

The backend selection mechanism exists below the generic `config::Configuration` interface. The prototype therefore does not need separate application pipelines for each backend.

Conceptually:



The primary prototype follows the left-hand path.

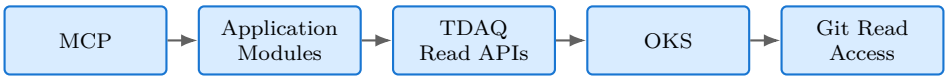
41 Module 34 – Read-Only Security Boundary

41.1 Purpose

The entire prototype is designed as a read-only system. This is an architectural property, not merely a user-interface feature.

41.2 Allowed Path

The allowed trust flow is:



41.3 Forbidden Paths

The architecture must not contain:

Forbidden operations include MCP-triggered commits, pushes, or configuration mutation; direct LLM access to infrastructure; and any application path that modifies production configuration.

42 Primary Data Flows

The architecture can also be understood as three major data flows.

42.1 Flow A — Historical Resolution

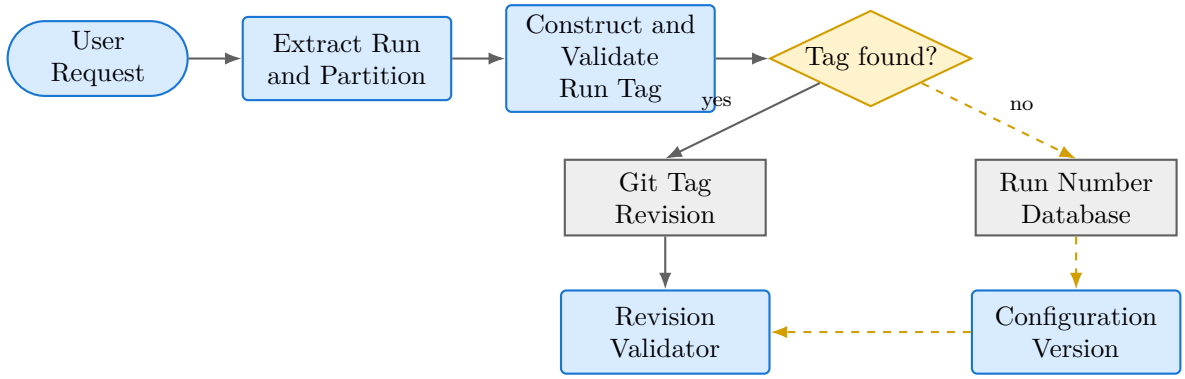


Figure 2: Historical run-to-revision resolution with explicit fallback

42.2 Flow B — Query Generation

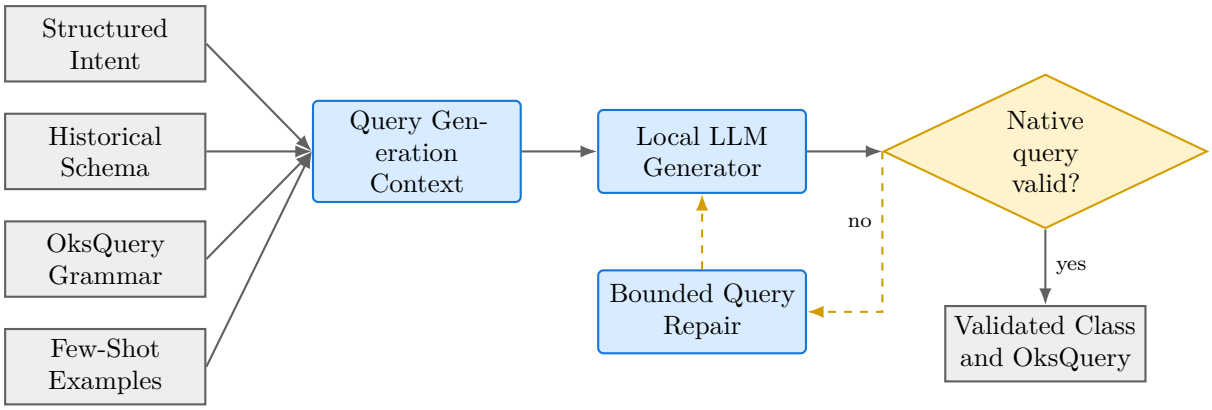


Figure 3: Schema-grounded query generation and bounded validation loop

42.3 Flow C — Query Execution and Answer

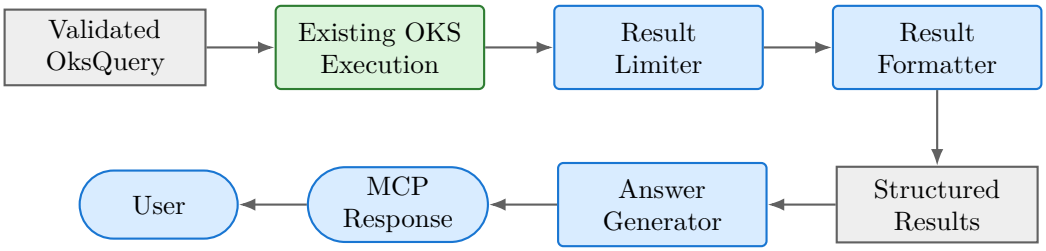


Figure 4: Validated query execution and response generation

43 Architectural Separation of Responsibilities

The most important separation in the system is between semantic intelligence and infrastructure execution.

This separation prevents the LLM from becoming a privileged infrastructure operator.

Area	Responsible Component
Natural-language interpretation	Intent Extractor / Local LLM
Run-to-revision mapping	Historical Configuration Resolver
Revision security validation	Revision / Version Validator
Configuration loading	Existing TDAQ configuration stack
Git checkout	Existing OKS Git integration
Schema retrieval	Existing Configuration APIs
Semantic query generation	Local LLM / OksQuery Generator
Query syntax/schema validation	Native `OksQuery.good()`
Query execution	Existing `OksClass::execute_query()`
Result serialization	Result Formatter
Natural-language response	Answer Generator / Local LLM
Protocol delivery	MCP Server
Error propagation	Central Error Handler

Table 1: Responsibility separation

44 Existing Versus New Components

The prototype intentionally maximizes reuse.

44.1 Existing Components

The following components are reused:

- `libpyconfig`;
- `config::Configuration`;
- `oksconfig`;
- `OksConfiguration`;
- `OksKernel`;
- existing Git integration;
- existing configuration schema APIs;
- `OksQuery`;
- `OksQuery.good()`;
- `OksClass::execute_query()`;
- existing typed configuration object accessors.

44.2 New Application Components

The prototype needs to implement:

- Intent Extractor;
- Historical Configuration Resolver glue;
- Revision / Version Validator;
- Schema Context Builder;
- Query Generation Context;
- OksQuery Generator;
- Bounded Query Repair;
- Result Limiter;
- Result Formatter;
- Answer Generator;
- MCP Server shell;
- Central Error Handler.

The architecture should avoid creating replacements for existing TDAQ/OKS components.

45 Explicitly Excluded Components

The following functionality is outside the prototype scope:

- configuration mutation;
- `commit()`;
- `abort()`;
- `set_config_version()`;
- RDB writer functionality;
- custom HTTP adapter;
- predictive maintenance;
- automatic configuration modification;
- custom XML parser;
- custom Git checkout implementation;
- custom OksQuery parser;
- custom OksQuery validator;
- custom configuration engine;
- semantic/vector retrieval.

These exclusions are deliberate. They prevent the six-week prototype from expanding into a replacement for the existing TDAQ/OKS infrastructure.

46 Core Architectural Principle

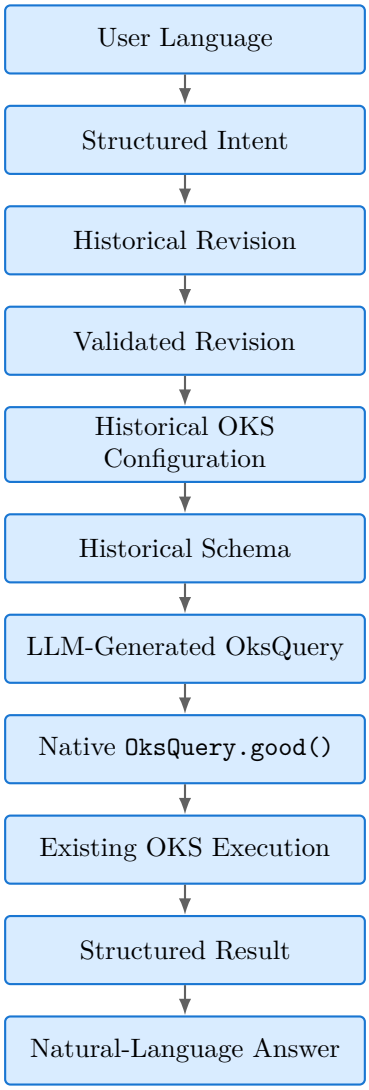
The final architecture follows one central principle:

The LLM adds semantic intelligence around the existing TDAQ/OKS configuration system; it does not replace the TDAQ/OKS configuration engine.

The LLM is responsible for understanding the user's language and constructing a candidate query. The existing TDAQ/OKS infrastructure remains responsible for:

- historical configuration access;
- repository resolution;
- schema representation;
- query validation;
- query execution;
- configuration object representation.

Therefore, the trust chain for a query is:



This architecture minimizes duplicated infrastructure, keeps historical configuration resolution deterministic, and ensures that generated queries are ultimately validated and executed by the native OKS system.